Lecture 6

15 October 2026 17:38

1.C.1 the first code

```
# Consider the following toy example
torch.manual_seed(1337)

B, T, C = 4, 8, 2 # batch, time, channels
x = torch.randn(B, T, C)
x.shape
```

1.C.2 calculation mean of the previous elements using loop---> this is a slow process

```
# We want mean of x[b, t} = mean_{i<=t} x[b, i]
xbow = torch.zeros((B, T, C))

for b in range(B):
    for t in range(T):
        xprev = x[b, :t+1]
        xbow[b, t] = torch.mean(xprev, 0)
```

1.C.3 how tensor or we can say matrix multiplication helps this process speed-up

```
torch.manual_seeds(42)

a = torch.ones(3, 3)
b = torch.randint(0, 10, (3, 2)).float()   -----> randint = takes random integer dtype values between 0 to 10, atlast, store it in the form of float in the tensor

c = a @ b

print("a=")
print(a)

print("b=")
print(b)

print( c )
```

Output:
a will be a 3 by 3 tensor of all one ---> like a identity matrix
b will be the random integer matrix of 3 by 2

c will be a 3 by 2 matrix form by multiplication of a matrix and b matrix

The is a way we can do the average using matrix ---i.e. as we need to do average of the previous random integer matrix b elements

1.C.4 THE MATHEMATICAL TRICK OF THIS IS using (tril) = lower triangular

```
torch.manual_seeds(42)

a = torch.tril( torch.ones(3, 3) ) ------> the change
A = a / torch.sum(a, 1, keepdim=True)
b = torch.randint(0, 10, (3, 2)).float()   -----> randint = takes random integer dtype values between 0 to 10, atlast, store it in the form of float in the tensor

c = a @ b

print("a=")
print(a)

print("b=")
print(b)

print( c )
```

V IMP:
Explanation: 1. we have change the identity matrix (a of ones) into a lower triangular matrix
2. and, we have converted that a matrix, in a form that, we will get average of the previous elements of the matrix b

Output:

```
a=
tensor([[1.0000, 0.0000, 0.0000],    -----> first row sum's to 1
        [0.5000, 0.5000, 0.0000],        -------> second row also sum's to 1, it take place only when it is divided in to 2 parts  i.e.  1 / 2 parts seperation
        [0.3333, 0.3333, 0.3333]])      ------> third row also sum's to 1, it take place only when it is divided in to 3 parts  i.e.  1 / 3 parts seperation
                                                  ..................and so on.......
b=
tensor([[2., 7.],
        [6., 4.],                                  This will be the input batch we will provided later
        [6., 5.]])

tensor([[2.0000, 7.0000],                 ------> the first two element average is the same element
        [4.0000, 5.5000],                  ---------> average of 2 and 6 gives = 4.000 and average of 4 and 7 gives = 5.5
        [4.6667, 5.3333]])                ..... And so one.....
```

V IMP
Now let's make the a and b in the real form

2.C.1 making the a matrix

wei = torch.tril (torch.ones(T, T)) ------wei = is called the a matrix, and as we are taking one-one batch with 8 tokens and later 65 sub probabilities the shape will be
(T, T) i.e. (8, 8)
wei = wei / wei.sum(1, keepdim=True) ------> combining in one single dimension
xbow2 = wei @ x # (B, T, T) @ (B, T, C) ---> (B, T, C)

Torch.all_close(xbow, xbow2, atol=1e-6)
the reason why we are using atol, cause the absolute tolerance is about 0.00000001 by default, our calculated value is smaller than that hence, hence, we need 1e-6 i.e. upto 6 decimal small value will be considered 0.000001

Output:
True

3.C.1 the final version which we will used for self-attention

Tril = torch.tril (torch.ones(T, T))
Wei = torch.zeros (T, T)

Wei = torch.masked_fill (tril == 0, float("-inf"))
Wei = F.softmax(wei, dim=-1)

Masked_fill takes two input parameter (cond, value)
Whenever, the condition is True, replace it by value

Where, -inf = is called -infinity

1. So now, the wei will look like this:

V.IMP

--------> this -inf (-infinity after softmax gives exact 0.0 value, which we need in the a matrix created at the first, i.e. the whole row value should be sum to 1

```
tensor([[0., -inf, -inf, -inf, -inf, -inf, -inf, -inf],
        [0., 0., -inf, -inf, -inf, -inf, -inf, -inf],
        [0., 0., 0., -inf, -inf, -inf, -inf, -inf],
        [0., 0., 0., 0., -inf, -inf, -inf, -inf],
        [0., 0., 0., 0., 0., -inf, -inf, -inf],
        [0., 0., 0., 0., 0., 0., -inf, -inf],
        [0., 0., 0., 0., 0., 0., 0., -inf],
        [0., 0., 0., 0., 0., 0., 0., 0.]])
```

2. Now, after softmax the above wei will look like this

```
tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.5000, 0.5000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.3333, 0.3333, 0.3333, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.2500, 0.2500, 0.2500, 0.2500, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.2000, 0.2000, 0.2000, 0.2000, 0.2000, 0.0000, 0.0000, 0.0000],
        [0.1667, 0.1667, 0.1667, 0.1667, 0.1667, 0.1667, 0.0000, 0.0000],
        [0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.0000],
        [0.1250, 0.1250, 0.1250, 0.1250, 0.1250, 0.1250, 0.1250, 0.1250]])
```

So, for calculating the just average of the previous tokens are this:

VERSION 1:

```
# Version 1: using simple loop, to calculate the avg of the previous tokens
# We want mean of x[b, t} = mean_{i<=t} x[b, i]
xbow = torch.zeros((B, T, C))

for b in range(B):
    for t in range(T):
        xprev = x[b, :t+1]
        xbow[b, t] = torch.mean(xprev, 0)
```

VERSION 2:

```
# Version 2: using Matrix, to calculate the avg of the previous tokens
wei = torch.tril(torch.ones(T, T))  # The reason why we took (T, T) is we are having total 8 token to calculate it's average
wei = wei / wei.sum(1, keepdim=True)

# V IMP
# So we are multiplying (T, T) to (B, T, C) i.e. (8, 8) @ (4, 8, 2) but, B is the total different batch, pytorch will not take it so we have to pass same batch dimension also with (T, T)
# So it will be (B, T, T) @ (B, T, C) = which will give (B, T, C) matrix of the average info, and shape will be same of (B, T, C)

xbow2 = wei @ x # (B, T, T) @ (B, T, C) = (B, T, C)
torch.allclose(xbow, xbow2, atol=1e-6)  # atol (Looser tolerance) = 0.000001
```

VERSION 3:

```
# Version 3: using softmax
```

```
tril = torch.tril(torch.ones(T, T))
wei = torch.zeros((T, T))
wei = wei.masked_fill(tril == 0, float("-inf"))
wei = F.softmax(wei, dim=-1)
```

The reason why the version 3 is good:

1. Cause the wei are initially made up of all zeros,

So we can refer them like initially they are starting with zero, and then will gain values

2. The -inf helps that the zero value are -inf, later softmaxing will convert to 0.0 and which is require for the sum total to 1 like the a matrix each row

Version 3: complete meaning

Line 1: tril = torch.tril(torch.ones(T, T))

Creates a lower triangular matrix of 1s of shape (8, 8):

text
```
[[1, 0, 0, 0, 0, 0, 0, 0],
 [1, 1, 0, 0, 0, 0, 0, 0],
 [1, 1, 1, 0, 0, 0, 0, 0],
 [1, 1, 1, 1, 0, 0, 0, 0],
 [1, 1, 1, 1, 1, 0, 0, 0],
 [1, 1, 1, 1, 1, 1, 0, 0],
 [1, 1, 1, 1, 1, 1, 1, 0],
 [1, 1, 1, 1, 1, 1, 1, 1]]
```

V.IMP

- **Lower triangle (1s):** Past and present tokens — allowed to attend.
- **Upper triangle (0s):** Future tokens — must be blocked.

------> REAL MEANING WHY WE ARE converting into 1 and 0

Line 2: wei = torch.zeros((T, T))

Creates a blank (8, 8) matrix of all zeros:

text
```
[[0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0],
 ...
 [0, 0, 0, 0, 0, 0, 0, 0]]
```

IMP STATEMENT : Think of wei as the **attention weights** — how much each token pays attention to every other token. Starting at zero means equal raw scores everywhere.

Line 3: wei = wei.masked_fill(tril == 0, float("-inf"))

Replaces every future position (upper triangle) with $-\infty$:

text
```
[[ 0,   -inf, -inf, -inf, -inf, -inf, -inf, -inf],
 [ 0,    0,   -inf, -inf, -inf, -inf, -inf, -inf],
 [ 0,    0,    0,   -inf, -inf, -inf, -inf, -inf],
 ...
 [ 0,    0,    0,    0,    0,    0,    0,    0 ]]
```

- 0 positions = past/present tokens = **allowed** to be attended to.
- -inf positions = future tokens = **completely blocked**.

Line 4: wei = F.softmax(wei, dim=-1)

Softmax converts the raw scores into **probabilities along each row** (dim=-1):

- $e^{0} = 1.0$ (for present/past positions)
- $e^{-\infty} = 0.0$ (for future positions — completely killed!)

After Softmax:

V.IMP

text
```
Row 0: [1.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00] ← only sees itself
Row 1: [0.50, 0.50, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00] ← sees tokens 0 and 1
Row 2: [0.33, 0.33, 0.33, 0.00, 0.00, 0.00, 0.00, 0.00] ← sees tokens 0, 1, 2
...
Row 7: [0.12, 0.12, 0.12, 0.12, 0.12, 0.12, 0.12, 0.12] ← sees all 8 tokens
```

This is **identical** to Version 2 where we divided by row sums! Softmax just does it more elegantly in one step.

Line 5: xbow3 = wei @ x

Matrix multiplies the attention weights (T, T) with original token embeddings (B, T, C):

text
(8, 8) @ (4, 8, 2) → (4, 8, 2)
For each token position, this computes the **weighted average of all past token embeddings** using the weights from wei.

Line 6: torch.allclose(xbow, xbow3, atol=1e-6)

Confirms that this Softmax version gives the **exact same result** as the original slow for-loop version!

Why is Version 3 THE most important?

This is where Karpathy is leading you! In real Self-Attention, the wei matrix is NOT fixed zeros — it is **dynamically computed** from the actual token content using **Query** and **Key** vectors:

text
Version 3 (now): wei = zeros → masked_fill → softmax
(fixed equal weights — dumb averaging)

Self-Attention (next): wei = Query @ Key.T → masked_fill → softmax
(learned dynamic weights — smart attention!)

The **only difference** between this averaging trick and full Self-Attention is **where wei comes from**.
Everything else — the masked_fill, softmax, and wei @ x — stays exactly the same!